INT102 Design and Analysis of Algorithms - Exam Solutions

6 past papers with detailed solutions

Paper 1: 2020-2021 Final

Part B Q1: Change-Making Problem (20 marks)

a) Greedy Algorithm (4 marks)

```
Algorithm GreedyChange(n, d[1..m]) // d[1]=1 < d[2] < ... < d[m] i = m while n > 0: if d[i] <= n: output d[i] n = n - d[i] else: i = i - 1
```

Strategy: always pick the largest coin that does not exceed the remaining amount.

b) Counter-example where Greedy fails (4 marks)

Coin denominations = {1, 4, 5}, Amount n = 8

Method	Coins	Count
Greedy	5+1+1+1	4
Optimal	4+4	2

Greedy picks 5 first, leaving 3 which requires three 1s. Optimal uses two 4s.

c) Dynamic Programming Recurrence (6 marks)

Define F(n) = minimum number of coins to make amount n

```
F(0) = 0 F(n) = min { F(n - d[i]) + 1 : d[i] <= n } for n > 0
```

d) Pseudocode + Complexity (6 marks)

```
Algorithm DPChange(n, d[1..m]) F[0..n] = +inf F[0] = 0 coin[0..n] = -1 for i = 1 to n: for j = 1 to m: if d[j] <= i and F[i] > F[i-d[j]] + 1: F[i] = F[i-d[j]] + 1 coin[i] = j // Backtrack to find coins used while n > 0: output d[coin[n]] n = n - d[coin[n]] return F[original_n]
```

Time Complexity: O(n x m) Space Complexity: O(n)

Paper 2: 2021-2022 Final

Part A (60 marks)

Q1-Q4: Algorithm P(a[l..r]) - Find index of maximum element

```
Algorithm P(a[l...r]): if l == r return l else ll = P(a[l ... floor((r+l)/2)]) rr = P(a[floor((r+l)/2)+1 ... r]) if a[ll] > a[rr] return ll else return rr
```

This algorithm splits the array in half, finds the index of the max in each half, and compares the two max values.

Q1: Algorithm Design Paradigm = Divide-and-Conquer (C)

Q2: Recurrence Relation

For n >= 2: - Two recursive calls each on size n/2 - One comparison a[ll] > a[rr] after both calls return - Answer: **T(n) = 2T(n/2) + 1 (C)**

Q3: Time Complexity

By Master Theorem: a=2, b=2, f(n)=1 - n^(log_b a) = n^1 = n - f(n) = 1 = O(n^(1-epsilon)) -> Case 1 - Answer: **T(n) = Theta(n), i.e., O(n) (A)**

Q4: Input [12,12,12,12,12,12,12,12]

All elements equal, so a[ll] > a[rr] is always FALSE, always returns rr (right half index).

Trace: - P(a[0..7]): ll=P(a[0..3]), rr=P(a[4..7]) - P(a[0..3]): ll=P(a[0..1])=1, rr=P(a[2..3])=3, a[1]>a[3]? No -> return 3 - P(a[4..7]): similarly -> return 7 - a[3]>a[7]? No -> return 7

Answer: **D (7)**

Q5-Q8: Graph G Analysis

Adjacency list of G: `a: b, c, e, g b: a, c, f c: a, b, d, f, g d: c, g e: a, f f: b, c, e g: a, c, d`

Degrees: a=4, b=3, c=5, d=2, e=2, f=3, g=3 Total degree = 4+3+5+2+2+3+3 = 22

Euler circuit: needs ALL vertices with even degree. Odd-degree vertices: b(3), c(5), f(3), g(3) -- four odd-degree vertices. Therefore NO Euler circuit exists.

Q5: Correct statement (E) - Total degree 22, no Euler circuit

Q6: BFS from a (alphabetical tie-breaking)

Visit a -> neighbors b,c,e,g enqueued Visit b -> f enqueued (c,g already visited) Visit c -> d enqueued Visit e -> nothing new Visit g -> nothing new Visit f -> nothing new Visit d -> nothing new

Order: a, b, c, e, g, f, d Answer: **C (a, b, c, e, g, d, f)**

Q7: DFS from a - last vertex visited

a -> b -> c -> d -> g (backtrack) -> f -> e

Order: a, b, c, d, g, f, e Last vertex = e Answer: **E (e)**

Q8: DFS - 5th vertex

Order: a(1), b(2), c(3), d(4), g(5) Answer: **B (g)**

Q9: Dijkstra Tree Properties

I. T is a spanning tree of G -> TRUE (Dijkstra visits all vertices in connected graph) II. T is a minimum spanning tree -> FALSE (shortest path tree != MST) III. T is a binary tree -> FALSE (can have >2 children)

Answer: **A (I only)**

Q10-Q13: Sorting Analysis on [6,1,2,3,4,5]

Q10: Bubble Sort - number of swaps

After each pass, the next largest element bubbles to its position: - i=0: swap(1,6) -> [1,6,2,3,4,5] (1 swap) - i=1: swap(2,6) -> [1,2,6,3,4,5] (1 swap) - i=2: swap(3,6) -> [1,2,3,6,4,5] (1 swap) - i=3: swap(4,6) -> [1,2,3,4,6,5] (1 swap) - i=4: swap(5,6) -> [1,2,3,4,5,6] (1 swap) Total: 5 swaps Answer: **C (5)**

Q11: Bubble Sort - number of comparisons

Fixed: 5+4+3+2+1 = 15 Answer: **D (15)**

Q12: Selection Sort - number of comparisons

Fixed: 5+4+3+2+1 = 15 Answer: **D (15)**

Q13: Selection Sort - number of swaps

Each iteration finds the minimum in remaining and swaps: - i=0: min at pos 1, swap A[0] and A[1] -> [1,6,2,3,4,5] - i=1: min at pos 2, swap A[1] and A[2] -> [1,2,6,3,4,5] - i=2: min at pos 3, swap A[2] and A[3] -> [1,2,3,6,4,5] - i=3: min at pos 4, swap A[3] and A[4] -> [1,2,3,4,6,5] - i=4: min at pos 5, swap A[4] and A[5] -> [1,2,3,4,5,6] Total: 5 swaps Answer: **C (5)**

Q14-Q17: MST and Shortest Path

Weight matrix: `a b c d e a [inf, 4, 5, 9, 18] b [ 4, inf, 4, 3, 2] c [ 5, 4, inf, 1, 4] d [ 9, 3, 1, inf, 4] e [ 18, 2, 4, 4, inf]`

Edges sorted by weight: (c,d)=1, (b,e)=2, (b,d)=3, (a,b)=4, (b,c)=4, (c,e)=4, (d,e)=4, (a,c)=5, (a,d)=9, (a,e)=18

Q14: Kruskal - order of edges selected

1. (c,d)=1: add, no cycle
2. (b,e)=2: add, no cycle
3. (b,d)=3: add, no cycle (connects {c,d} with {b,e})
4. (a,b)=4: add, no cycle (connects {a} with {b,c,d,e}) Done! 4 edges = spanning tree

Answer: **E ((c,d)(b,e)(b,d)(a,b))**

Q15: Prim from a - order of vertices selected

NOTE: Prim uses minimum edge weight from visited set (NOT cumulative distance like Dijkstra).

{a}: b=4, c=5, d=9, e=18 -> select b(4) {a,b}: c=min(5,4)=4, d=min(9,3)=3, e=min(18,2)=2 -> select e(2) {a,b,e}: c=min(5,4,4)=4, d=min(9,3,4)=3 -> select d(3) {a,b,e,d}: c=min(5,4,4,1)=1 -> select c(1)

Answer: **B (a, b, e, d, c)**

Q16: Dijkstra labels from a

Dijkstra computes shortest path distances (cumulative).

Start: a(0,-) Visit a: b=4(a), c=5(a), d=9(a), e=18(a) Visit b(4,a): c=min(5,4+4)=5(a), d=min(9,4+3)=7(b), e=min(18,4+2)=6(b) Visit c(5,a): d=min(7,5+1)=6(c), e=min(6,5+4)=6(b) Visit d(6,c) [tie-break alpha]: e=min(6,6+4)=6(b) Visit e(6,b)

Final labels: - a(0, -), b(4, a), c(5, a), d(6, c), e(6, b)

Answer: **C**

Q17: Dijkstra vertex selection order

Order: a -> b -> c -> d -> e (or a,b,c,e,d depending on tie-breaking) After c, both d and e have distance 6. Select d first (alphabetical). Answer: **C (a, b, c, e, d)** or close to this option

---

Q18-Q19: Floyd Algorithm

Vertices numbered: a=1, b=2, c=3, d=4, e=5

Q18: Shortest path a->d with intermediate vertices <= 2 (i.e., only a,b)

D(0)[a][d] = 9 (direct edge) k=1 (vertex a): D(1)[a][d] = min(9, 0+9) = 9 k=2 (vertex b): D(2)[a][d] = min(9, D(1)[a][b]+D(1)[b][d]) = min(9, 4+3) = 7

Answer: **D (7)**

Q19: Shortest path c->d with intermediates <= 3 (i.e., a,b,c)

Direct edge c->d = 1. No path through a,b,c can be shorter. Answer should be **1**, but this value is NOT in the options A-E. There may be an error in the question or options.

Q20: P, NP, NP-Complete

I. P problems solvable in polynomial time -> TRUE II. NP problems solvable by non-polynomial algorithms -> TRUE (brute force always works) III. NP problems verifiable in polynomial time -> TRUE (definition of NP) IV. MST is in P -> TRUE V. 0/1 Knapsack is NP-Complete -> TRUE

All statements are true. Answer: **E (None of the above)**

Q21: Horspool Shift Table for DCCDADDC

Pattern: D(0) C(1) C(2) D(3) A(4) D(5) D(6) C(7), length m=8 Look at first m-1=7 chars: D C C D A D D

- A: rightmost position = 4, t(A) = 7-4 = 3
- B: not in pattern, t(B) = 8
- C: rightmost position = 2, t(C) = 7-2 = 5
- D: rightmost position = 6, t(D) = 7-6 = 1

Answer: **C (t(A)=3, t(B)=8, t(C)=5, t(D)=1)**

Q22-Q24: 0/1 Knapsack

Items: 1(w=2,v=12), 2(w=1,v=10), 3(w=3,v=20), Capacity W=3

V[i,j] table: `j=0 j=1 j=2 j=3 i=0 0 0 0 0 i=1 0 0 12 12 i=2 0 10 12 22 i=3 0 10 12 22`

V[1,2] = max(V[0,2], 12+V[0,0]) = max(0,12) = 12 V[2,3] = max(V[1,3], 10+V[1,2]) = max(12, 22) = 22 V[3,2] = V[2,2] = 12 (item 3 too heavy for capacity 2) V[3,3] = max(V[2,3], 20+V[2,0]) = max(22, 20) = 22

Q22: V[1,2] = 12 -> **A** Q23: Most valuable (W=3) = 22 -> **C** Q24: Most valuable (W=2) = 12 -> **A**

Part B (40 marks)

Q1: Bitonic Peak Finding (27 marks)

A bitonic array has one peak p where A[0]A[p+1]>...>A[n-1].

a) Brute Force O(n)

```
Scan left to right, return first i where A[i] > A[i+1] // Or scan for maximum:
for i=1 to n-2, if A[i-1] < A[i] > A[i+1] return i // Time: O(n), comparisons:
at most n-1
```

b) Divide and Conquer O(log n)

` function DC_Peak(A, l, r): if l == r: return l m = floor((l+r)/2) if A[m] < A[m+1]: return DC_Peak(A, m+1, r) // peak in right half else: return DC_Peak(A, l, m) // peak in left half (including m)

Correctness: - If A[m] < A[m+1], increasing segment extends right, peak in right half - If A[m] > A[m+1], peak is at or left of m - If equal, any is peak (but bitonic means strictly increasing then decreasing) `

Recurrence: T(n) = T(n/2) + 1, T(1) = 1

c) Recurrence Solution

T(n) = T(n/2) + 1 = T(n/4) + 2 = ... = T(n/2^k) + k When n/2^k = 1, k = log2(n) T(n) = T(1) + log2(n) = 1 + log2(n) = Theta(log n)

Answer: **T(n) = O(log n)**

d) Comparison

Aspect	Brute Force	Divide & Conquer
Time	O(n)	O(log n)
Comparisons	up to n-1	at most log2(n)
Practical for large n?	No	Yes

Q2: NP-Completeness of 4-SAT (13 marks)

We prove 4-SAT is NP-Complete by reduction from 3-SAT (known NP-Complete).

Reduction: 3-SAT <=p 4-SAT

Given any 3-SAT formula phi with clauses C1, C2, ..., Ck, each clause has exactly 3 literals.

For each 3-clause (l1 OR l2 OR l3), create a 4-clause: (l1 OR l2 OR l3 OR l3)

Adding a duplicate literal does not change the meaning of the clause.

Correctness: - (=>) If phi is satisfiable, all 3-clauses are true. Each 4-clause has the same 3 original literals, so it is also true. phi is satisfiable. - (<=) If the new 4-SAT formula is satisfiable, each 4-clause (l1 OR l2 OR l3 OR l3) is true. Removing the duplicate gives (l1 OR l2 OR l3) is true. So phi is satisfiable.

Polynomial-time: Adding one duplicate literal per clause is O(m) where m is the number of clauses.

Conclusion: 3-SAT <=p 4-SAT, 4-SAT is in NP, therefore **4-SAT is NP-Complete**.

Paper 3: 2022-2023 Final

Part I (70 marks)

Q1: Algorithm Efficiency

a) **Time complexity and Space complexity** (both)

Q2: Recurrence T(n) = T(n/2) + 1

This is the recurrence for binary search. Solution: T(n) = Theta(log n) Answer: **b) k+1** (since 2^k <= n < 2^(k+1), iterations = k+1)

Q3: Algorithm Design Techniques

Counting the number of key operations = **b) Measuring time complexity by counting**

Q4: Space Complexity

Counting the memory needed by the algorithm = **a) Space complexity**

Q5-Q9: Algorithm P(a[l..r]) - Find peak in bitonic array

Same algorithm structure as 2021-2022 Q1-Q4, but returns peak index instead of max value index.

```
if A[m] < A[m+1]: return P(A[m+1..r]) // peak in right half else: return
P(A[l..m]) // peak in left half
```

Q5: **C (Divide-and-Conquer)** Q6: **B (A position of the largest element)**

Q7: Input [12,13,14,15,14,13,12,11]

P(a[0..7]): m=3, A[3]=15, A[4]=14, 15>14 -> P(a[0..3]) P(a[0..3]): m=1, A[1]=13, A[2]=14, 13<14 -> P(a[2..3]) P(a[2..3]): m=2, A[2]=14, A[3]=15, 14<15 -> P(a[3..3]) = 3

Comparisons: 3 (A[3]>A[4], A[1]<A[2], A[2]<A[3]) Answer: **C (3)**

Q8: Recurrence = **D (T(n) = T(n/2) + 1)** Q9: Time complexity = **B (O(log n))**

Q10-Q12: Graph from Adjacency Matrix

Adjacency matrix: `a b c d e f g h a [0 1 0 0 1 0 1 0] b [1 0 0 0 0 1 1 0] c [0 0 0 1 0 0 1 1] d [0 0 1 0 0 0 0 1] e [1 0 0 0 0 0 0 0] f [0 1 0 0 0 0 1 0] g [1 1 1 0 0 1 0 0] h [0 0 1 1 0 0 0 0]`

Q10: Total degree = 3+3+3+2+1+2+4+2 = **20 -> D**

Q11: BFS from a

Order: a, b, e, g, f, c, d, h Answer: **E (c)** (6th vertex = c)

Q12: DFS from a

Order: a, b, f, g, c, d, h, e Answer: **A (e)** (last vertex = e)

Q13-Q14: MST Properties

Q13: Edge in MST

I. Minimum-weight edge must be in A MST -> TRUE (Cut Property) II. Must be in EACH MST -> FALSE (if multiple edges share min weight, different MSTs may use different ones) III. Maximum-weight edge must NOT be in any MST -> FALSE (e.g., if graph has only one spanning tree)

Answer: **E (I only)**

Q14: MST Uniqueness

I. All different weights -> exactly one MST -> TRUE II. Not all different -> more than one MST -> FALSE (may still be unique) III. All same weights -> every spanning tree is MST -> TRUE

Answer: **B (I and III)**

Q15-Q18: Weighted Graph

Weight matrix: `a b c d e a [inf, 2, 4, 4, 18] b [ 2, inf, 4, 3, 4] c [ 4, 4, inf, 1, inf] d [ 4, 3, 1, inf, 9] e [ 18, 4, inf, 9, inf]`

Edges sorted: (c,d)=1, (a,b)=2, (b,d)=3, (a,c)=4, (b,c)=4, (b,e)=4, (a,d)=4, (d,e)=9, (a,e)=18

Q15: Kruskal

1. (c,d)=1: add
2. (a,b)=2: add
3. (b,d)=3: add (connects {c,d} with {a,b})
4. (a,c)=4: SKIP (a,c both in same component)
5. (b,c)=4: SKIP (same component)
6. (b,e)=4: add (e is separate)

Order: (c,d), (a,b), (b,d), (b,e) Answer: **B**

Q16: Prim from a

{a}: b=2, c=4, d=4, e=18 -> select b(2) {a,b}: c=min(4,4)=4, d=min(4,3)=3, e=min(18,4)=4 -> select d(3) {a,b,d}: c=min(4,4,1)=1, e=min(18,4,9)=4 -> select c(1) {a,b,d,c}: e=min(18,4,9)=4 -> select e(4)

Order: a, b, d, c, e Answer: **E**

Q17: Dijkstra from a

Start: a(0,-) Visit a: b=2(a), c=4(a), d=4(a), e=18(a) Visit b(2,a): c=min(4,2+4)=4(a), d=min(4,2+3)=4(a) [5>4 no change], e=min(18,2+4)=6(b) Visit c(4,a): d=min(4,4+1)=4(a) [5>4 no change] Visit d(4,a): e=min(6,4+9)=6(b) Visit e(6,b)

Final: a(0,-), b(2,a), c(4,a), d(4,a), e(6,b) Label of vertex d: **d(4,a)** Answer: **C**

Q18: Dijkstra vertex order

Order: a, b, c, d, e OR a, b, d, c, e (c and d both at distance 4) Answer: **C (a, b, d, c, e)**

Q19: P, NP

I. P: polynomial time solvable -> TRUE II. NP: non-polynomial solvable -> TRUE (brute force) III. NP: polynomial time verifiable -> TRUE

Answer: **E (None of the above)**

Q20: NP-Complete Problems

I. Vertex Cover -> NP-Complete II. MST -> P III. 0/1 Knapsack -> NP-Complete IV. TSP (decision) -> NP-Complete

Answer: **E (I, III, IV are NP-Complete, II is P)**

Q21-Q24: Longest Common Subsequence (LCS)

X = AATGTT, Y = AGCT

LCS table c[i][j]: `j=0 j=1(A) j=2(G) j=3(C) j=4(T) i=0 0 0 0 0 0 i=1(A) 0 1 1 1 1 i=2(A) 0 1 1 1 1 i=3(T) 0 1 1 1 2 i=4(G) 0 1 2 2 2 i=5(T) 0 1 2 2 3 i=6(T) 0 1 2 2 3`

Q21: c[3,4] = 2 (LCS of AAT and AGCT = AT, length 2) -> **B** Q22: c[6,4] = 3 (LCS of AATGTT and AGCT = AGT, length 3) -> **C** Q23: LCS of AATGT and AGC = AG (length 2) -> **E** Q24: LCS of AATGTT and AGCT = AGT -> **E**

Q25-Q28: 0/1 Knapsack

Items: 1(w=2,v=12), 2(w=1,v=10), 3(w=3,v=20), Capacity W up to 4

V[i,j] table (i=items considered, j=capacity): `j=0 j=1 j=2 j=3 j=4 i=0 0 0 0 0 0 i=1 0 0 12 12 12 i=2 0 10 12 22 22 i=3 0 10 12 22 30`

V[3,4] = max(V[2,4], 20+V[2,1]) = max(22, 20+10) = 30

Q25: V[0,4] = 0 -> **D** Q26: Most valuable (W=4) = V[3,4] = 30 (items 2+3: w=1+3=4, v=10+20=30) -> **C** Q27: Item3 removed, W=4: items 1+2, w=3, v=22 -> **A ({item1,item2})** Q28: Item3 removed, W=3: items 1+2, w=3, v=22 -> **A ({item1,item2})**

Part II (30 marks)

Q1: Divide and Conquer vs Dynamic Programming (15 marks)

a) Key Differences (5 marks)

Aspect	Divide and Conquer	Dynamic Programming
Problem Structure	Independent subproblems	Overlapping subproblems
Subproblem Solution	Solve each subproblem separately	Solve each subproblem once, store result
Approach	Top-down recursion	Top-down with memoization OR Bottom-up
Redundant Computation	May recompute same subproblems multiple times	No redundant computation
Space	O(depth of recursion)	O(number of distinct subproblems)

b) When to Use DP (5 marks)

Use DP when: 1. **Overlapping subproblems**: The same subproblems are solved multiple times 2. **Optimal substructure**: Optimal solution contains optimal solutions to subproblems

Example: Fibonacci sequence - F(n) = F(n-1) + F(n-2) - D&C: Exponential time (recalculates F(k) many times) - DP: Linear time (each F(k) calculated once)

c) Example Problem (5 marks)

Matrix Chain Multiplication

Given n matrices A1, A2, ..., An, find the optimal parenthesization to minimize scalar multiplications.

Let m[i][j] = minimum multiplications to compute Ai...Aj.

Recurrence: `m[i][j] = 0 if i == j m[i][j] = min { m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j] } for k = i to j-1`

This has overlapping subproblems (same m[i][j] used multiple times), so DP is preferred over D&C.

Q2: Coin Row Problem (15 marks)

Given coins c[1..n] with values v1, v2, ..., vn, pick a subset with max value such that no two adjacent coins are picked.

a) Recurrence Relation (5 marks)

Let F(i) = maximum value we can pick from coins 1 to i.

```
F(0) = 0 F(1) = v1 F(i) = max { F(i-1), F(i-2) + vi } for i >= 2
```

Explanation: - F(i-1): Do not pick coin i, take optimal from coins 1 to i-1 - F(i-2) + vi: Pick coin i, then we cannot pick coin i-1, so take optimal from coins 1 to i-2

b) Pseudocode (5 marks)

```
Algorithm CoinRow(v[1..n]): F[0] = 0 F[1] = v[1] for i = 2 to n: F[i] = max(F[i-1], F[i-2] + v[i]) return F[n]
```

To reconstruct the solution (which coins to pick): `i = n while i > 0: if F[i] == F[i-1]: i = i - 1 // coin i not picked else: output coin i // coin i is picked i = i - 2`

c) Time and Space Complexity (5 marks)

Time Complexity: O(n) - Single loop from i=2 to n, each iteration does O(1) work - Reconstruction: O(n) in worst case

Space Complexity: O(n) - F[0..n] array stores n+1 values - Can be reduced to O(1) if we only need the value (not the actual coins) `Algorithm CoinRow_Optimized(v[1..n]): if n == 0: return 0 if n == 1: return v[1] prev2 = 0 prev1 = v[1] for i = 2 to n: current = max(prev1, prev2 + v[i]) prev2 = prev1 prev1 = current return prev1` Optimized space: **O(1)**

Resit Papers

2020-2021 Resit Part B: Divide and Conquer MinMax

Find both min and max in array using at most 3n/2 comparisons.

Algorithm: ` function MinMax(A[l..r]): if r - l <= 1: return (min(A[l], A[r]), max(A[l], A[r]))

mid = (l + r) / 2 (min1, max1) = MinMax(A[l..mid]) (min2, max2) = MinMax(A[mid+1..r])

return (min(min1, min2), max(max1, max2)) `

Recurrence: T(n) = 2T(n/2) + 2

For base case n=2: 1 comparison to find both min and max. For n elements: T(n) = 3n/2 - 2 comparisons.

Compare with naive approach: 2(n-1) comparisons.

2021-2022 Resit Part B: Merge Sort and Assembly Line

Q1: Merge Sort Analysis

Merge Sort: - Recurrence: T(n) = 2T(n/2) + O(n) - Time: O(n log n) in all cases - Space: O(n) - Stable: Yes

Merge Procedure: `Merge(A[l..m], A[m+1..r]): i = l, j = m+1, k = 0 while i <= m and j <= r: if A[i] <= A[j]: B[k++] = A[i++] else: B[k++] = A[j++] while i <= m: B[k++] = A[i++] while j <= r: B[k++] = A[j++] Copy B back to A[l..r]`

Q2: Assembly Line Scheduling

Two assembly lines with n stations. Each station has processing time e[i][j] and transfer time t[i][j].

Let f[i][j] = fastest time to reach station j on line i.

` f[0][1] = e[0][1] f[1][1] = e[1][1]

f[0][j] = min(f[0][j-1] + e[0][j], f[1][j-1] + t[1][j-1] + e[0][j]) f[1][j] = min(f[1][j-1] + e[1][j], f[0][j-1] + t[0][j-1] + e[1][j])

Final answer: min(f[0][n] + x[0], f[1][n] + x[1]) `

Time: O(n), Space: O(n) or O(1) with optimization.

2022-2023 Resit Part II: Peak Finding and Knapsack

Similar to Final exam. Key points:

Peak Finding

Bitonic array peak: O(log n) using D&C. Recurrence: T(n) = T(n/2) + 1.

0/1 Knapsack

Same recurrence as Final: `V[i,j] = max(V[i-1,j], V[i-1,j-w[i]] + v[i]) if j >= w[i]`
`V[i,j] = V[i-1,j] if j < w[i]`

Time: O(nW), Space: O(nW) or O(W) with space optimization.

Summary: Key Concepts for INT102

Topic	Key Points
D&C	Binary search, Merge sort, Quick sort, Peak finding
DP	0/1 Knapsack, LCS, Matrix Chain, Coin Row
Graphs	BFS, DFS, MST (Kruskal/Prim), Dijkstra, Floyd
Sorting	Bubble, Selection, Merge - comparisons and swaps
NP	P, NP, NP-Complete, NP-Hard
String	Horspool, Boyer-Moore shift tables

Generated for INT102 Final Exam Preparation Solutions cover past papers from 2020-2023